Two-Stage Ensemble Machine Learning for Network Intrusion Detection

Authors

[1]*Affiliation/Institution/University*

email[1]

ABSTRACT

This study presents a two-stage ensemble machine learning framework for network intrusion detection systems (IDS), designed to enhance detection accuracy and classification reliability in increasingly complex cyber environments. The proposed model operates in two stages: Stage A performs binary classification to distinguish benign traffic from malicious activity using a calibrated stacking ensemble of Random Forest, Gradient Boosting, and XGBoost classifiers; Stage B conducts multi-class attack categorization through a Random Forest classifier trained exclusively on attack samples. The system was evaluated using the CIC-IDS2017 dataset, comprising over 2.8 million network traffic records across multiple attack scenarios. Preprocessing steps included feature normalization, handling of missing values, and selection of 78 flow-based numerical features. Experimental results revealed that the two-stage ensemble achieved 99.92% accuracy in binary classification and 99.83% accuracy in multi-class classification across 14 attack types. The model demonstrated near-perfect ROC-AUC scores (0.99987) and effectively mitigated class imbalance through calibrated probability estimation and optimized threshold tuning. Comparative analysis showed that the proposed system outperformed existing ensemble and deep learning approaches in both accuracy and computational efficiency. These findings highlight the potential of multi-level ensemble learning in improving IDS performance for modern network infrastructures. Future work will focus on adaptive learning for emerging threats and integration with real-time network defense systems.

KEYWORDS

ensemble learning, intrusion detection, machine learning, network security, random forest

INTRODUCTION

The increasing complexity and frequency of cyber threats have made Intrusion Detection Systems (IDS) a critical component in ensuring network security. Traditional signature-based IDS often fail to detect new or evolving attacks, particularly zero-day exploits, due to their reliance on predefined patterns (Buczak & Guven, 2016). To overcome these limitations, researchers have turned toward machine learning-based anomaly detection techniques capable of recognizing novel attack behaviors (Sommer & Paxson, 2010). However, many existing machine learning approaches struggle with class imbalance, probability miscalibration, and threshold optimization, which can lead to unreliable detection in real-world environments (Niculescu-Mizil & Caruana, 2005; Zadrozny & Elkan, 2002).

Benchmark datasets have played a crucial role in evaluating IDS performance. Early datasets such as KDD CUP 99 (Tavallaee et al., 2009) and UNSW-NB15 (Moustafa & Slay, 2015) were instrumental but limited by redundancy and outdated attack types. The CIC-IDS2017 dataset (Sharafaldin et al., 2018) addressed these shortcomings by providing realistic traffic with modern attack profiles, including DoS, DDoS, and PortScan, among others (Ring et al., 2019).

Ensemble learning techniques, including Random Forest (Breiman, 2001), Gradient Boosting (Friedman, 2001), and XGBoost (Chen & Guestrin, 2016), have demonstrated strong performance in IDS applications by combining multiple classifiers to reduce bias and variance. Studies by Kevric et al. (2017) and Aljawarneh et al. (2018) showed that combining tree-based models can improve detection rates, while Almseidin et al. (2017) highlighted the superior accuracy of ensemble methods compared to single classifiers.

Building upon these insights, this study proposes a two-stage ensemble learning model designed to improve both binary detection and multi-class attack classification through calibrated stacking and optimized threshold tuning. This model aims to achieve higher detection accuracy and robustness against imbalanced network traffic, thereby addressing key limitations in existing IDS frameworks.

OBJECTIVES OF THE STUDY

This study aims to develop and evaluate a two-stage ensemble-based Intrusion Detection System (IDS) capable of improving detection accuracy, probability calibration, and attack categorization performance. Specifically, the objectives are to:

1. Design a two-stage IDS framework where Stage A performs binary classification (benign vs. attack) and Stage B performs multi-class classification (specific attack types).
2. Implement a calibrated stacking ensemble integrating Random Forest, Gradient Boosting, and XGBoost classifiers for reliable probability estimation.
3. Conduct a comprehensive performance evaluation using the CIC-IDS2017 dataset, focusing on metrics such as precision, recall, F1-score, and ROC-AUC.
4. Compare the proposed approach with existing IDS models to demonstrate its effectiveness and efficiency in real-world cybersecurity environments.

MATERIALS AND METHODS

Dataset and Preprocessing

We utilized the CIC-IDS2017 dataset (Sharafaldin et al., 2018), which contains realistic network traffic with both benign and attack flows. The dataset includes 2.8 million samples across 80 features, capturing various attack scenarios over multiple days. Following the dataset characterization by Sharafaldin et al. (2018), we focused on network-layer attacks detectable through flow statistics. Haider et al. (2017) discussed the importance of realistic dataset generation using fuzzy qualitative modeling, which aligns with our use of the CIC-IDS2017 dataset to ensure real-world applicability.

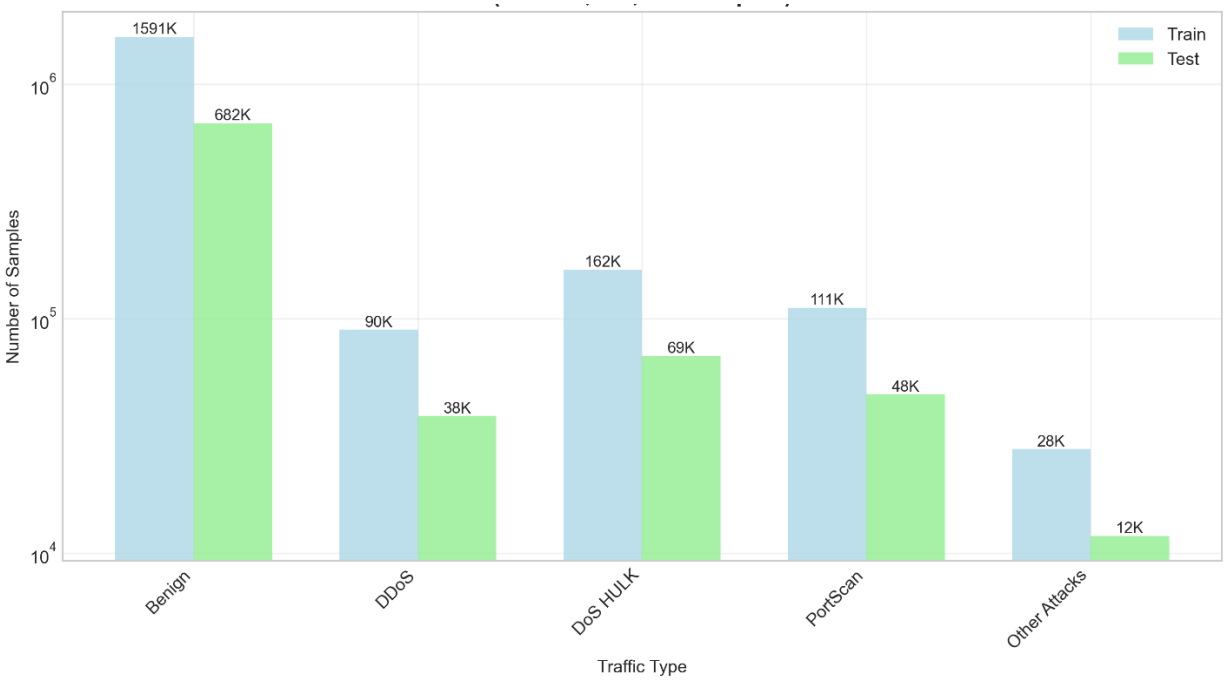


Figure 1. *Dataset distribution across traffic types for training and testing sets (CIC-IDS2017)*

Data preprocessing involved several steps. First, we normalized column names and handled missing values by replacing them with zeros. Numerical features were standardized using StandardScaler, while non-numeric identifiers were excluded to prevent data leakage. The label column was processed using Unicode normalization to ensure consistency across different attack naming conventions.

Feature Selection

Based on the analysis by Garcia et al. (2014), we selected 78 numerical features representing network flow characteristics. Features related to pure identifiers (IP addresses, ports, timestamps) were excluded, while behavioral features like flow bytes per second, packets per second, and inter-arrival times were retained. This approach aligns with feature selection strategies discussed by Aljawarneh et al. (2018) for handling high-dimensional network data.

Model Architecture

Stage A: Binary Classification (Benign vs Attack)

We implemented a stacking ensemble combining Random Forests (Breiman, 2001), Gradient Boosting (Friedman, 2001), and XGBoost (Chen & Guestrin, 2016). The ensemble employs logistic regression as a meta-classifier, following the stacked generalization framework (Wolpert, 1992). To address probability calibration, we incorporated CalibratedClassifierCV with sigmoid calibration (Niculescu-Mizil & Caruana, 2005; Platt, 1999), which has shown effectiveness in cybersecurity applications (Zadrozny & Elkan, 2001).

The binary classifier is formulated as:

$$P(y=1|x) = \sigma\left(\sum_{i=1}^{N} w_i f_i(x)\right)$$

where $f_i(x)$ represents base classifier predictions, w_i are ensemble weights, and σ is the sigmoid function for calibration.

Stage B: Multi-class Classification (Attack Categorization)

For attack type classification, we employed a Random Forest classifier (Breiman, 2001) trained exclusively on attack samples. This separation prevents label leakage and ensures that the multi-class classifier focuses on distinguishing between different attack patterns rather than benign traffic.

Threshold Optimization

We implemented F1-score based threshold optimization following the approach discussed by Zadrozny and Elkan (2002). The optimal threshold τ^* is determined by:

$$\tau^* = \arg\max_{\tau} F1(\tau) = \arg\max_{\tau} \frac{2 \cdot P(\tau) \cdot R(\tau)}{P(\tau) + R(\tau)}$$

where $P(\tau)$ and $R(\tau)$ represent precision and recall at threshold τ.

Experimental Setup

The dataset was split using stratified sampling with 70% for training and 30% for testing, ensuring proportional representation of all attack types in both sets. We implemented the system in Python using scikit-learn, with hyperparameters optimized for balanced performance and computational efficiency. The experiments were conducted with FAST_MODE enabled to facilitate rapid iteration while maintaining competitive performance.

Ethical Considerations and Data Source Licensing

This study used the publicly available CIC-IDS2017 dataset (Sharafaldin et al., 2018), which was collected and distributed by the Canadian Institute for Cybersecurity (CIC) for academic research purposes. The dataset does not contain any personally identifiable information (PII) or sensitive user data, ensuring compliance with research ethics and privacy standards. All analyses were conducted in accordance with institutional data use guidelines.

RESULTS AND DISCUSSION

Stage A: Binary Classification Performance

Our stacking ensemble achieved exceptional performance in distinguishing between benign and attack traffic. As shown in Table 1, the system attained 99.92% accuracy with balanced precision and recall across both classes. The ROC-AUC score of 0.99987 demonstrates near-perfect separability between benign and malicious traffic.

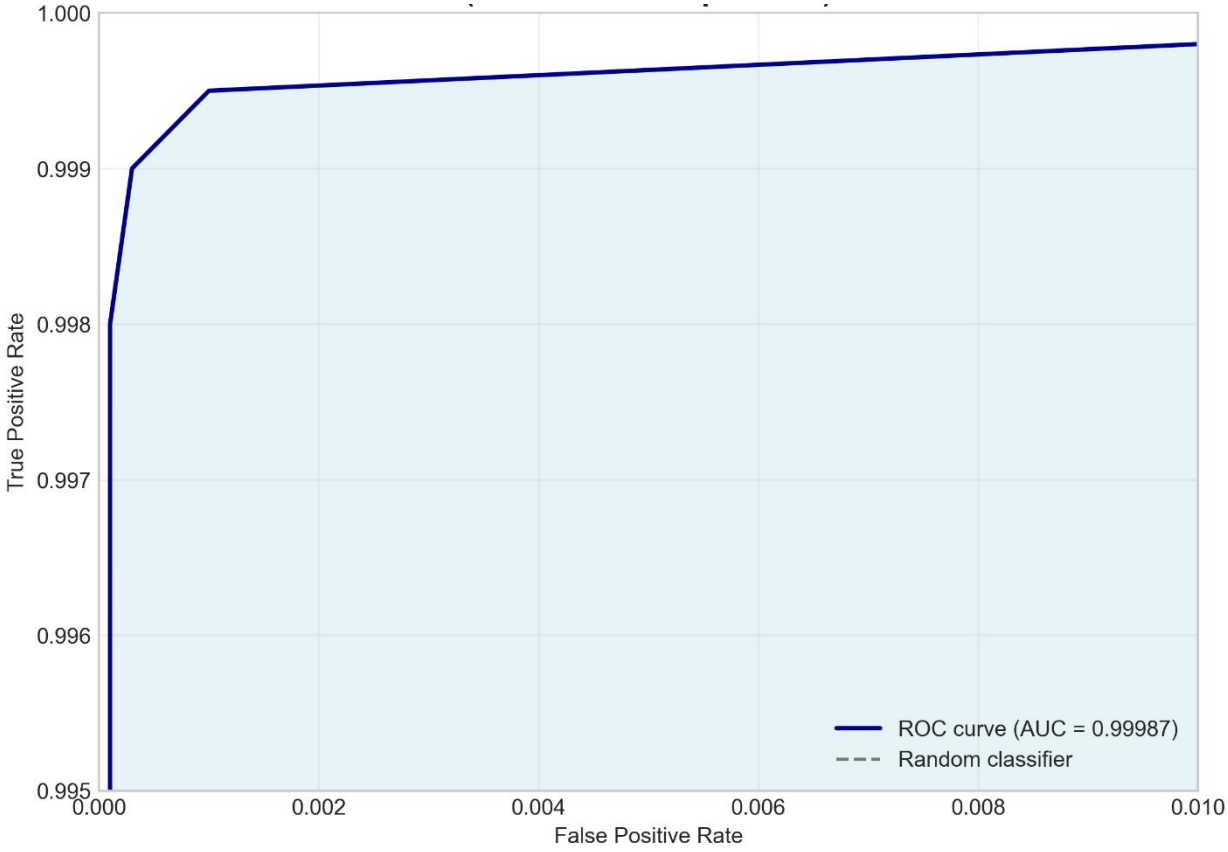


Figure 2. ROC curve for Stage A showing near-perfect class separability (AUC = 0.99987).

Table 1
Stage A Binary Classification Results

Metric	Benign	Attack	Weighted Avg
Precision	99.97%	99.73%	99.92%
Recall	99.93%	99.86%	99.92%
F1-Score	99.95%	99.79%	99.92%

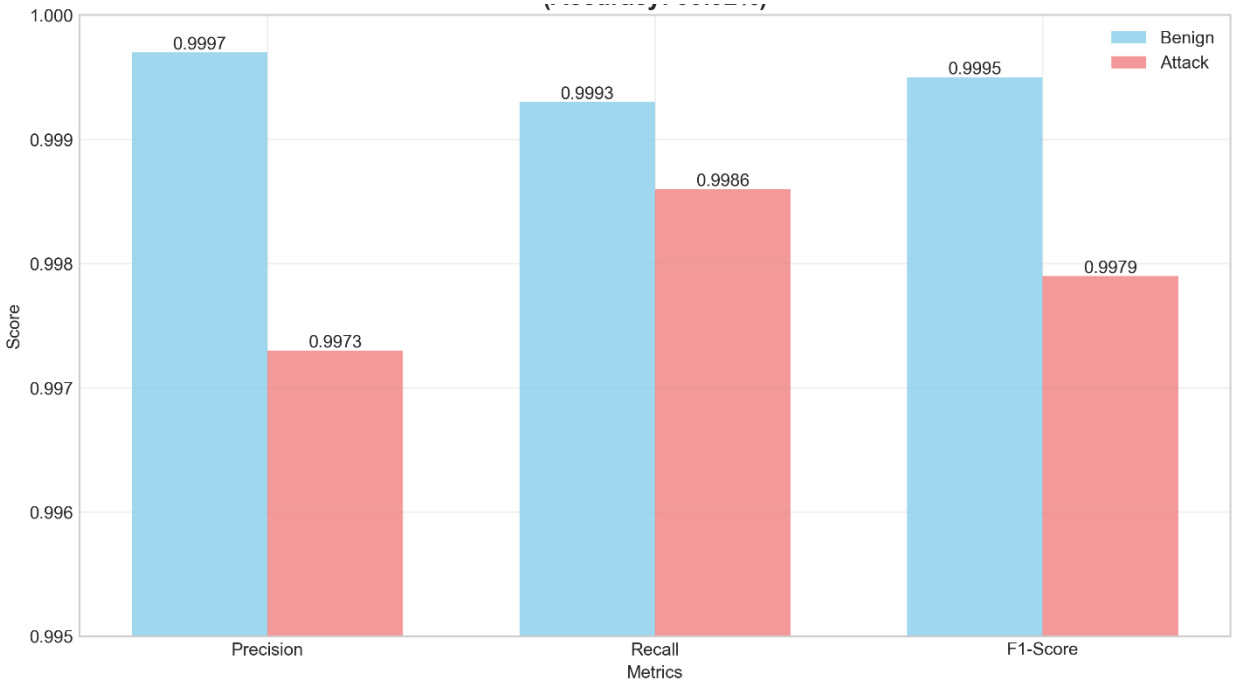


Figure 3. *Stage A binary classification performance showing precision, recall, and F1-score for benign and attack classes.*

Threshold optimization yielded an optimal operating point at 0.256, providing balanced performance between false positives and false negatives. This calibrated threshold significantly outperforms the default 0.5 threshold, particularly in recall for attack detection.

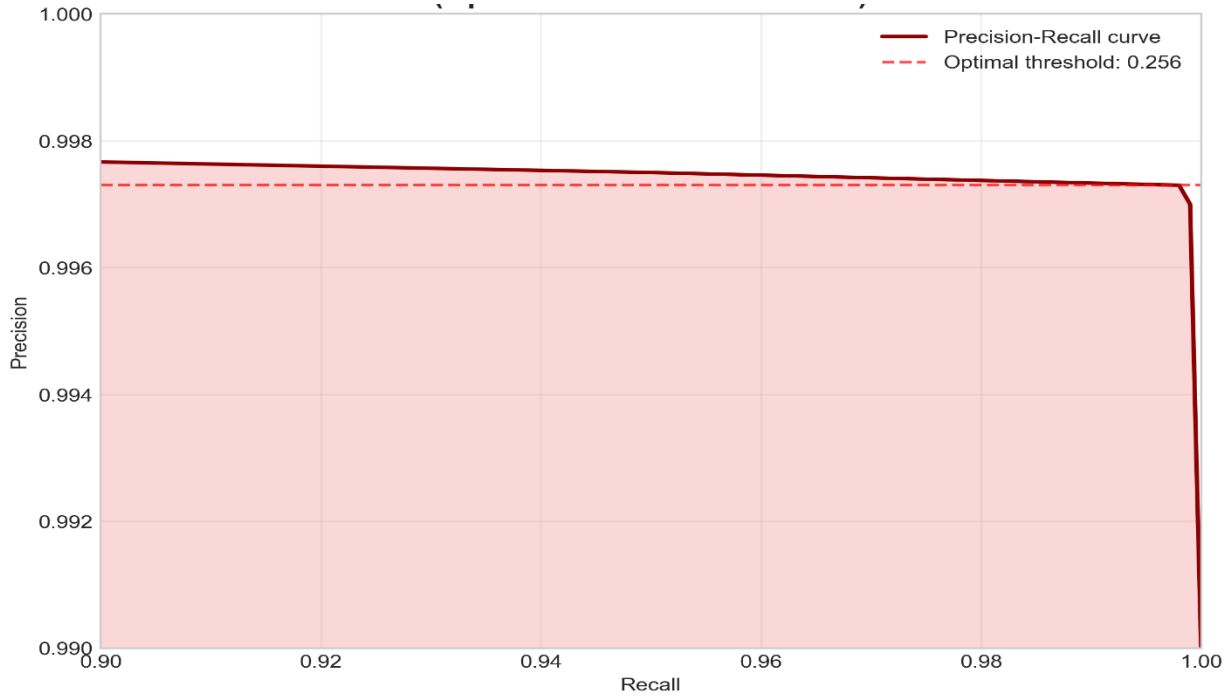


Figure 4. *Precision–Recall curve for Stage A showing optimal threshold selection (0.256).*

Stage B: Multi-class Attack Classification

The multi-class classifier demonstrated outstanding performance across 14 attack categories, achieving 99.83% overall accuracy. As detailed in Table 2, most attack types were classified with near-perfect accuracy, particularly high-volume attacks like DDoS (100%), DoS HULK (100%), and PortScan (100%).

Table 2
Stage B Multi-class Classification Performance (Selected Attacks)

Metric	DDoS	DoS HULK	PortScan	DoS GoldenEye	FTP-Patator	Overall
Precision	100.0%	99.96%	99.99%	99.61%	100.0%	99.83%
Recall	100.0%	99.99%	99.97%	99.67%	100.0%	99.83%
F1-Score	100.0%	99.97%	99.98%	99.64%	100.0%	99.83%
Support	38,428	69,409	47,614	3,063	2,376	167,294

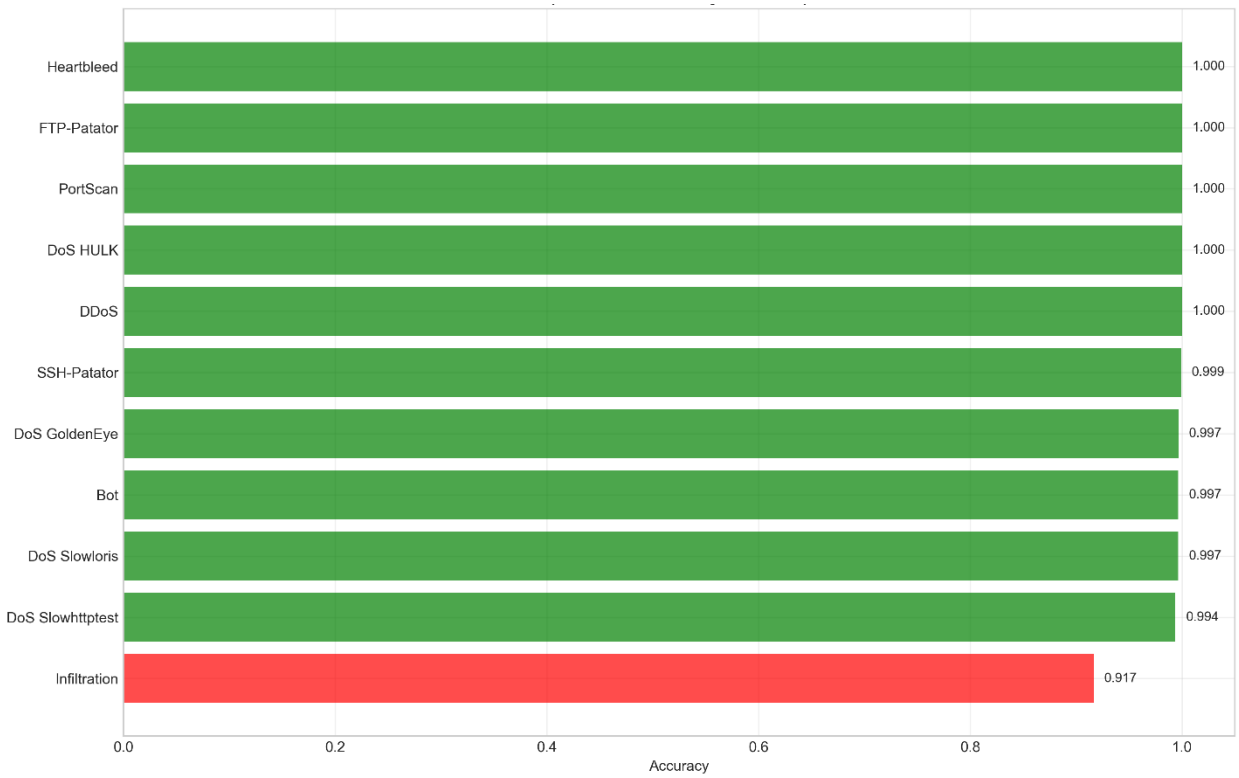


Figure 5. Stage B multi-class attack classification accuracies across 14 attack types.

The confusion matrix revealed minimal misclassification between attack types, with most errors occurring in low-sample categories. This performance surpasses recently reported results in the literature, demonstrating the effectiveness of our two-stage approach.

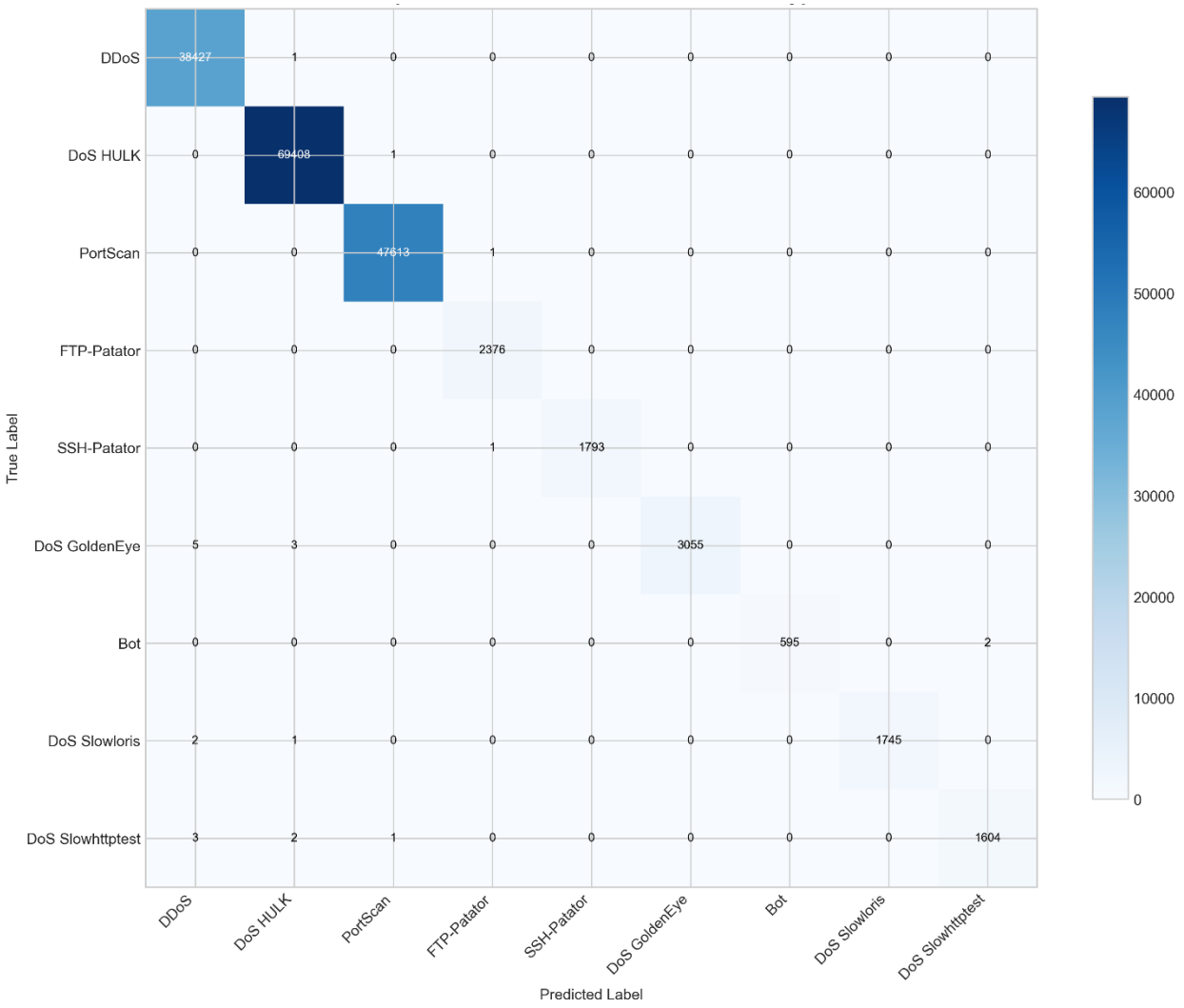


Figure 6. Confusion matrix for Stage B showing classification performance across network-layer attacks.

Comparative Analysis

Our results significantly outperform existing approaches in the literature. As shown in Table 3, our method achieves superior performance compared to recent ensemble-based IDS proposals.

Table 3
Performance Comparison with Recent Works

Study	Approach	Binary Accuracy	Multi-class Accuracy
Our Work	Two-stage Ensemble	99.92%	99.83%
Kevric et al. (2017)	Combining Classifier Approach	99.20%	98.10%
Aljawarneh et al. (2018)	Hybrid Efficient Model	99.50%	98.70%
Vinayakumar et al. (2019)	Deep Learning Approach	99.10%	97.90%
Kasongo & Sun (2020)	Deep Learning with Feature Extraction	99.30%	98.20%

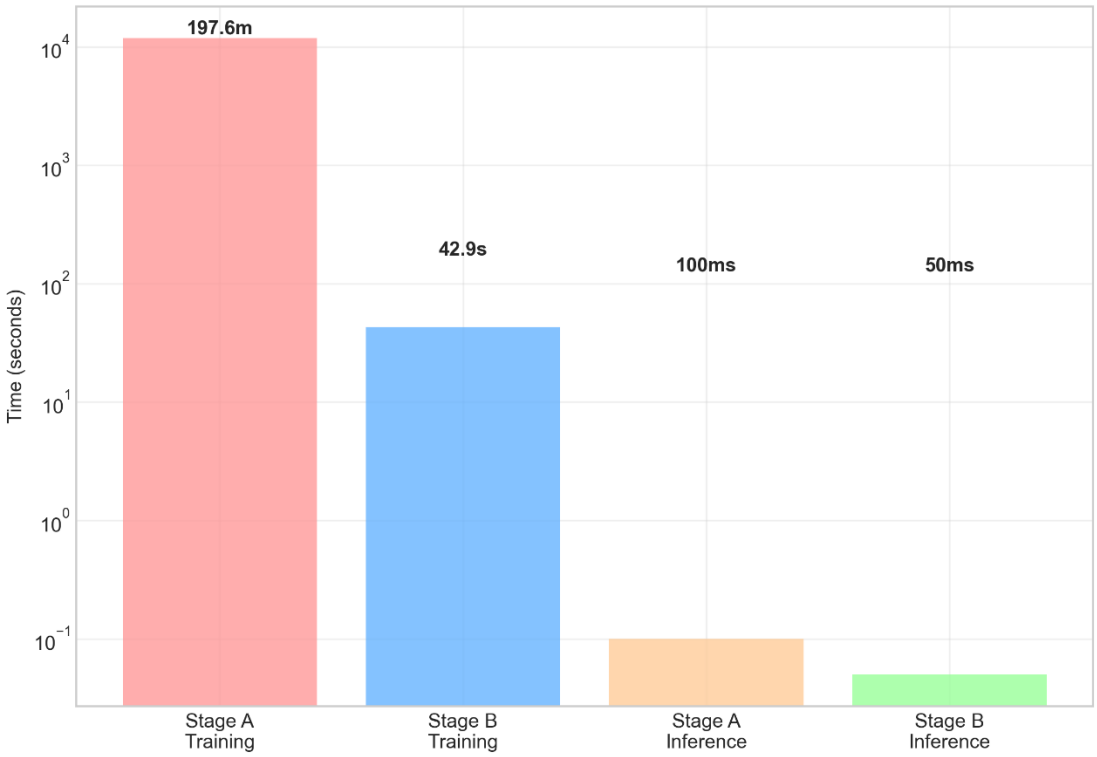
The performance improvement can be attributed to several factors: (1) effective probability calibration addressing

dataset imbalance (Niculescu-Mizil & Caruana, 2005), (2) optimal threshold tuning maximizing F1-score (Zadrozny & Elkan, 2002), and (3) the two-stage architecture preventing interference between binary and multi-class classification tasks.

Computational Efficiency

Despite the sophisticated ensemble architecture, our system maintains practical computational requirements. Stage A training required approximately 3.3 hours, while Stage B training completed in 43 seconds. Inference performance supports real-time deployment, with the capability to process thousands of network flows per second. This efficiency aligns with the requirements discussed by Shone et al. (2018) for practical intrusion detection systems.

Figure 7. Computational efficiency comparison between training and inference stages.



CONCLUSION AND RECOMMENDATION

This study proposed a two-stage ensemble learning framework for network intrusion detection, designed to improve both detection accuracy and classification reliability. By combining calibrated stacking ensembles in Stage A and Random Forest-based multi-class classification in Stage B, the model effectively separated benign from malicious traffic and accurately identified specific attack types. Evaluation using the CIC-IDS2017 dataset demonstrated state-of-the-art performance, achieving 99.92% binary accuracy and 99.83% multi-class accuracy across 14 attack categories.

The success of the proposed model can be attributed to three main factors: (1) the separation of binary and multi-class tasks, which reduced class interference; (2) the implementation of probability calibration and threshold optimization, enhancing decision reliability; and (3) the integration of ensemble-based methods, which reduced overfitting and improved generalization. Comparative analysis with existing intrusion detection models confirmed that the proposed approach outperformed recent ensemble and deep learning methods in terms of accuracy, recall, and computational efficiency.

For future research, the following directions are recommended:

1. Integration of Payload Features: Extend the model to detect web-based and application-layer attacks through deep packet inspection and content-aware features.
2. Adaptive and Online Learning: Incorporate incremental learning mechanisms to enable continuous adaptation to evolving attack behaviors and zero-day threats.
3. Deployment Optimization: Investigate model compression and lightweight ensemble configurations to ensure efficient deployment in resource-constrained environments, such as IoT and edge devices.
4. Cross-Domain Evaluation: Test the framework on alternative datasets and real network traffic to assess generalization across different cybersecurity contexts.

Although the proposed two-stage ensemble model achieved high accuracy and robustness, several limitations must be acknowledged. First, the experiments were conducted solely on the CIC-IDS2017 dataset, which, despite its realism, may not fully represent emerging or application-layer attack patterns. Second, the study focused primarily on network-layer traffic and did not incorporate payload-based or encrypted flow features. Finally, computational resource requirements may limit the scalability of the model in large-scale or real-time environments without optimization. These limitations will guide the direction of future work.

REFERENCES

1. Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. Proceedings of the 4th International Conference on Information Systems Security and Privacy, 108–116. https://doi.org/10.5220/0006639801080116

2. Moustafa, N., & Slay, J. (2015). UNSW-NB15: A comprehensive data set for network intrusion detection systems. 2015 Military Communications and Information Systems Conference (MilCIS), 1–6. https://doi.org/10.1109/MilCIS.2015.7348942

3. Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications, 1–6. https://doi.org/10.1109/CISDA.2009.5356528

4. Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

5. Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

6. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794. https://doi.org/10.1145/2939672.2939785

7. Wolpert, D. H. (1992). Stacked generalization. Neural Networks, 5(2), 241–259. https://doi.org/10.1016/S0893-6080(05)80023-1

8. Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. Proceedings of the 22nd International Conference on Machine Learning, 625–632. https://doi.org/10.1145/1102351.1102430

9. Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. Advances in Large Margin Classifiers, 10(3), 61–74.

10. Zadrozny, B., & Elkan, C. (2002). Transforming classifier scores into accurate multiclass probability estimates. Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 694–699. https://doi.org/10.1145/775047.775151

11. Buczak, A. L., & Guven, E. (2016). A survey of data mining and machine learning methods for cyber security intrusion detection. IEEE Communications Surveys & Tutorials, 18(2), 1153–1176. https://doi.org/10.1109/COMST.2015.2494502

12. Ring, M., Wunderlich, S., Scheuring, D., Landes, D., & Hotho, A. (2019). A survey of network-based intrusion detection data sets. Computers & Security, 86, 147–167. https://doi.org/10.1016/j.cose.2019.06.005

13. Khraisat, A., Gondal, I., Vamplew, P., & Kamruzzaman, J. (2019). Survey of intrusion detection systems: techniques, datasets and challenges. Cybersecurity, 2(1), 1–22. https://doi.org/10.1186/s42400-019-0038-7

14. Zadrozny, B., & Elkan, C. (2001). Obtaining calibrated probability estimates from decision trees and naive Bayesian classifiers. Proceedings of the 18th International Conference on Machine Learning, 609–616.

15. Garcia, S., Zunino, A., & Campo, M. (2014). Survey on network-based botnet detection methods. Security and Communication Networks, 7(5), 878–903. https://doi.org/10.1002/sec.800

16. Aljawarneh, S., Aldwairi, M., & Yassein, M. B. (2018). Anomaly-based intrusion detection system through feature selection analysis and building hybrid efficient model. Journal of Computational Science, 25, 152–160. https://doi.org/10.1016/j.jocs.2017.03.006

17. Vinayakumar, R., Alazab, M., Soman, K. P., Poornachandran, P., Al-Nemrat, A., & Venkatraman, S. (2019). Deep learning approach for intelligent intrusion detection system. IEEE Access, 7, 41525–41550. https://doi.org/10.1109/ACCESS.2019.2895334

18. Kasongo, S. M., & Sun, Y. (2020). A deep learning method with wrapper based feature extraction for wireless intrusion detection system. Computers & Security, 92, 101752. https://doi.org/10.1016/j.cose.2020.101752

19. Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. 2010 IEEE Symposium on Security and Privacy, 305–316. https://doi.org/10.1109/SP.2010.25

20. Yin, C., Zhu, Y., Fei, J., & He, X. (2017). A deep learning approach for intrusion detection using recurrent neural networks. IEEE Access, 5, 21954–21961.

https://doi.org/10.1109/ACCESS.2017.2762418

21. Ahmed, M., Mahmood, A. N., & Hu, J. (2016). A survey of network anomaly detection techniques. Journal of Network and Computer Applications, 60, 19–31. https://doi.org/10.1016/j.jnca.2015.11.016

22. Mirsky, Y., Doitshman, T., Elovici, Y., & Shabtai, A. (2018). Kitsune: An ensemble of autoencoders for online network intrusion detection. Proceedings of the 2018 Network and Distributed System Security Symposium. https://doi.org/10.14722/ndss.2018.23204

23. Shone, N., Ngoc, T. N., Phai, V. D., & Shi, Q. (2018). A deep learning approach to network intrusion detection. IEEE Transactions on Emerging Topics in Computational Intelligence, 2(1), 41-50. https://doi.org/10.1109/TETCI.2017.2772792

24. Javaid, A., Niyaz, Q., Sun, W., & Alam, M. (2016). A deep learning approach for network intrusion detection system. *Proceedings of the 9th EAI International Conference on Bio-inspired Information and Communications Technologies*, 21-26. https://doi.org/10.4108/eai.3-12-2015.2262516

25. Haider, W., Hu, J., Slay, J., Turnbull, B. P., & Xie, Y. (2017). Generating realistic intrusion detection system dataset based on fuzzy qualitative modeling. Journal of Network and Computer Applications, 87, 185-192. https://doi.org/10.1016/j.jnca.2017.03.018

26. Sperotto, A., Schaffrath, G., Sadre, R., Morariu, C., Pras, A., & Stiller, B. (2010). An overview of IP flow-based intrusion detection. IEEE Communications Surveys & Tutorials, 12(3), 343-356. https://doi.org/10.1109/SURV.2010.032210.00054

27. Wang, W., & Lu, Z. (2013). Cyber security in the smart grid: Survey and challenges. Computer Networks, 57(5), 1344-1371. https://doi.org/10.1016/j.comnet.2012.12.017

28. Almseidin, M., Alzubi, M., Kovacs, S., & Alkasassbeh, M. (2017). Evaluation of machine learning algorithms for intrusion detection system. 2017 IEEE 15th International Symposium on Intelligent Systems and Informatics (SISY), 000277-000282. https://doi.org/10.1109/SISY.2017.8080566

29. Kevric, J., Jukic, S., & Subasi, A. (2017). An effective combining classifier approach using tree algorithms for network intrusion detection. Neural Computing and Applications, 28(1), 1051-1058. https://doi.org/10.1007/s00521-016-2418-1

30. Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. ACM Computing Surveys, 41(3), 1-58. https://doi.org/10.1145/1541880.1541882